

DOC § 08 · ARCHITECTURE · TECHNICAL DUE-DILIGENCE

Luvion / Sui Integration

Threshold Ed25519 · FROST for Sui account-auth, with ML-DSA-65 retained as the long-term post-quantum migration layer.

Pin Sui mainnet-v1.70.2 · commit 6d4ec0b0621dd9555753c9ecd5be021b25a0d267
Crypto RFC 9591 FROST(Ed25519, SHA-512) · fastcrypto ZIP215 · FIPS 204 ML-DSA
Policy 22-of-33 off-chain threshold · single Ed25519 signer to Sui
Source luvion-protocol-core / docs/architecture / 08-sui-integration-plan.en.md

08 Sui Integration Technical Architecture

Status: technical due-diligence and architecture-review document for the Sui integration. This document focuses on technical facts, boundaries, implementable design, risks, and verification methods.

Maintenance rule: every change to this English version must be mirrored in `08-sui-integration-plan.zh.md` in the same change set.

1. Technical Conclusion

The near-term Sui integration uses **threshold Ed25519/FROST as the account-auth compatibility path**. Current `src/signing.rs` implements 22-of-33 threshold ML-DSA-65; Luvion keeps threshold ML-DSA as the long-term post-quantum migration layer and core technical moat, while adding an Ed25519/FROST transaction-signing backend on top of the existing orchestration, network envelope, VRF proof, and cluster runtime to stay compatible with Sui's current native transaction-auth mechanism.

Sui account-auth compatibility route:

- 33 Luvion nodes hold threshold key shares generated by DKG or controlled key ceremony; the ceremony keeps the full private key out of single-node memory, storage, and export flows.
- Any policy-approved set of 22 nodes cooperatively produces one standard 64-byte Ed25519 signature accepted by the Sui/fastcrypto ZIP215 verifier.
- Sui sees one normal Ed25519 signer and one normal Sui address.
- Luvion enforces the 22-of-33 policy off-chain; Sui's native verification logic stays stable.

Supporting technical choices:

- Keep ML-DSA-65 as Luvion's long-term post-quantum migration layer; track threshold ML-DSA standardization maturity, third-party audit, and production hardening separately.
- Keep the current Ed25519 VRF/network key for committee selection and network message authentication.
- Use a new FROST aggregate key for Sui account-auth.
- Position Move shared-vault as an optional application-layer custody module that only controls assets deposited into the vault; Sui account-auth is handled by the aggregate Ed25519 address.

Unified positioning: Luvion's Sui integration uses threshold Ed25519/FROST as the account-auth compatibility path while keeping threshold ML-DSA as the long-term post-quantum migration layer. This route fits Sui's current native transaction-auth mechanism and labels Sui account-auth separately from the long-term PQ migration path.

2. Goals and Scope

Goals:

- Support 22-of-33 Luvion signing for Sui `TransactionData` .
- Output a Sui-native Ed25519 signature envelope: `0x00 || ed25519_signature || aggregate_public_key` .
- Integrate Sui object/gas resolution, dry-run/simulation, submit, and effects/checkpoint watching.
- Preserve a clear owned/shared object boundary: low-latency signing uses the address-owned execution path, while public state transparency uses the shared-object path.
- Provide state machines, interfaces, test vectors, and failure-handling rules suitable for later audit.

Scope boundaries:

- Sui protocol and Sui validator signature verification stay unchanged.
- Sui user-signature compatibility uses Ed25519/FROST output.
- Luvion enforces the 22-of-33 policy off-chain.
- Optional Move vaults are application-layer custody modules.
- This document focuses on technical architecture, interfaces, state machines, risks, and verification methods.

3. Current Repository State

Reusable capabilities:

- 22-of-33 node orchestration, round coordination, retry, timing, and TCP cluster runtime.
- Per-node share loading, node identity checks, and request_id-scoped one-time round state.
- Network envelope signatures, VRF-style proofs, and base p2p/message routing modules.
- Complete experimental ML-DSA threshold signing path with rejection/retry operational experience.

Components to add for the Sui integration:

- Add an Ed25519/FROST share format, separate from the current `s1/s2/t0` polynomial shares.
- Add FROST Round1/Round2 wire DTOs for nonce commitments, partial signatures, partial verification, and blame records.
- Keep `gen_shares` scoped to local development/test fixture generation; testnet may use controlled import ceremony, while the production release gate is audited DKG or an independently reviewed secure key ceremony.
- Add a threshold Ed25519 backend while keeping current `ed25519-dalek` usage for network authentication and VRF proof.

4. Sui Compatibility Constraints

Sui Ed25519 user signature format:

```
signature_bytes = 0x00 || ed25519_signature_64 || ed25519_public_key_32
```

Sui Ed25519 address derivation:

```
sui_address = Blake2b256(0x00 || ed25519_public_key_32)
```

Sui transaction signing message:

```
intent_message = IntentMessage<TransactionData> {
  intent: Intent::sui_transaction(),
  value: transaction_data,
}

message_to_sign = Blake2b256(BCS(intent_message))
```

`Intent::sui_transaction()` resolves to `IntentScope::TransactionData` in Sui source. `Sender-SignedTransaction` is the authority-side scope for a user-signed transaction; the Sui user signature path uses `TransactionData`.

Required rules:

- The signing input must be the intent digest Sui verifies for account signatures.
- FROST Round2 signs this 32-byte Blake2b256 digest as the Ed25519 message bytes; Ed25519/FROST-Ed25519 still performs its own internal SHA-512 computation.
- PTB bytes remain the transaction payload; the signing layer uses the intent digest.
- Transaction/effects digests remain object, effects, and query identities; user signatures use the `IntentMessage<TransactionData>` digest.
- `sender` must equal the Sui address derived from the aggregate public key.
- If `gas_owner != sender`, the sponsored transaction still needs gas owner authorization.
- Sui native account authentication follows Sui-supported signature schemes; Luvion uses Ed25519/FROST to fit that boundary, while threshold ML-DSA advances as a separate long-term post-quantum migration layer.

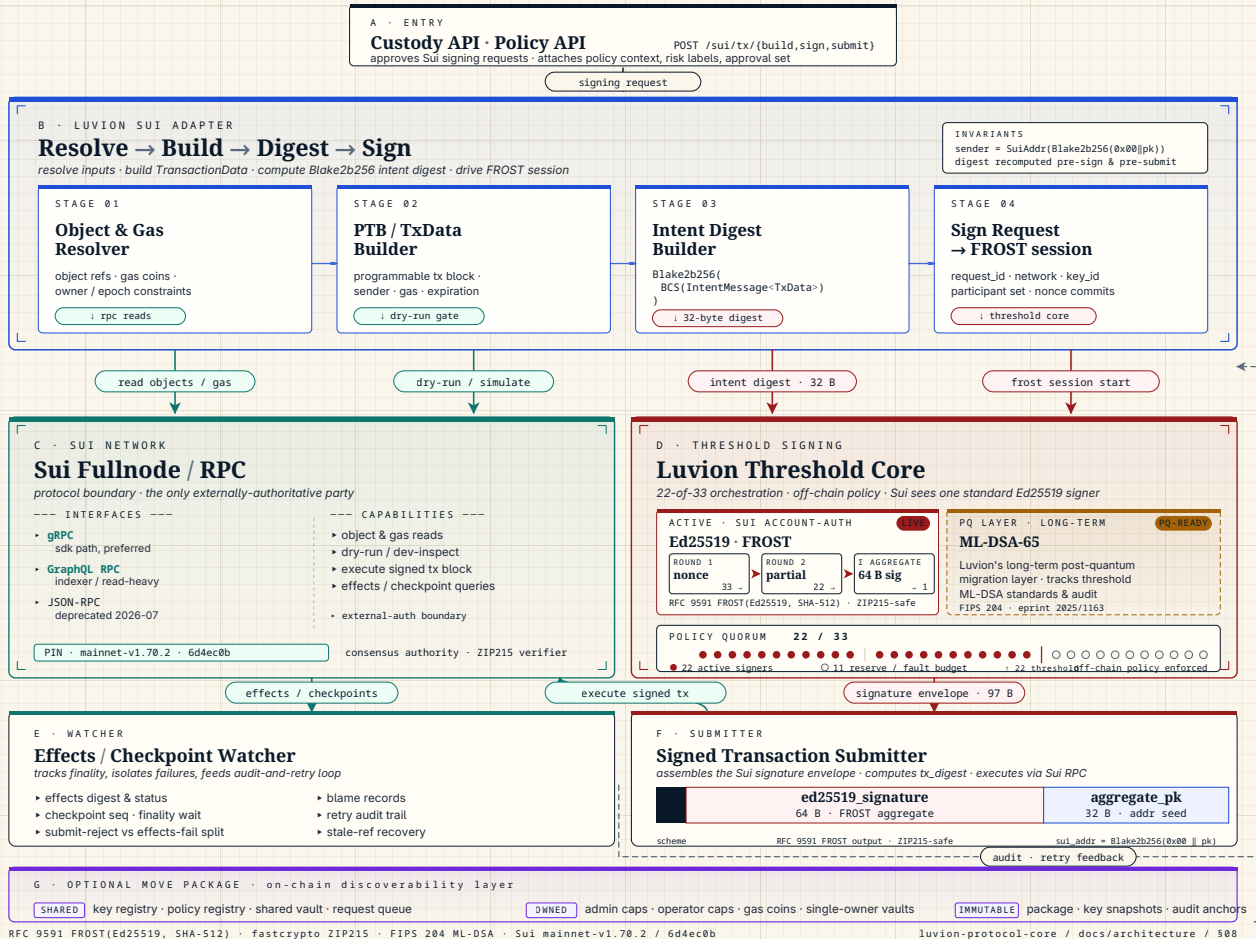
5. Architecture Overview

DOC 508 · ARCHITECTURE OVERVIEW · SUI MAINNET-V1.70.2 · 6D4EC0B

Luvion / Sui Integration

Threshold Ed25519 · FROST for Sui account-auth, with ML-DSA-65 retained as the long-term post-quantum migration layer.

LANES



6. Signer Mode Decision Matrix

MODE	22-OF-33	SUI TRANSACTION AUTH	INTEGRATION PLAN
Current ML-DSA-65 signer	Yes, inside Luvion	Kept as separate backend	Continue serving the ML-DSA/PQ-ready path.
Current Ed25519 usage	Single-signer	Network/VRF side	Continue serving VRF/network envelope.
Sui native multisig	Up to 10 keys	Sui-native support	Keep as Sui-native capability reference; use FROST for the 22-of-33 account-auth compatibility path.
Move shared-vault policy	Application-level policy	Gates shared-vault flows	Optional product-layer module.
Threshold Ed25519/FROST	Yes	Yes	Sui account-auth compatibility route.

The FROST backend must:

- Output a standard 64-byte Ed25519 signature.
- Output a 32-byte aggregate Ed25519 public key.
- Sui/fastcrypto Ed25519 verification follows ZIP215 semantics; select RFC 9591 `FROST(Ed25519, SHA-512)`, whose output satisfies the cofactored verification equation accepted by ZIP215.
- Verify FROST output against Sui fastcrypto ZIP215 test vectors before making a deployment-readiness claim.
- Use the same fastcrypto/ZIP215 verification mode for local self-verification.
- Bind each session to `request_id`, network, intent digest, aggregate public key, participant set, and nonce commitments.
- Verify partial signatures before aggregation and produce blame records on failure.
- Use per-session nonces; retries must use fresh nonces.

7. Core Object Definitions

Objects in this section are proposed additions to be implemented in the Sui/FROST modules.

7.1 Off-Chain Objects

`SuiSigningRequest`

FIELD	DESCRIPTION
<code>request_id: [u8; 32]</code>	Internal Luvion request ID, bound to round state and audit records.
<code>network: SuiNetwork</code>	<code>Mainnet</code> <code>Testnet</code> <code>Devnet</code> <code>Localnet</code> .
<code>sender: SuiAddress</code>	Must equal the address derived from the aggregate Ed25519 public key.
<code>gas_owner: SuiAddress</code>	May equal sender; otherwise sponsored transaction rules apply.
<code>transaction_data_bcs: Vec<u8></code>	BCS-encoded <code>TransactionData</code> .
<code>intent_scope: IntentScope</code>	User transactions use <code>TransactionData</code> scope.
<code>object_refs: Vec<ObjectRef></code>	<code>(ObjectID, SequenceNumber, ObjectDigest)</code> .
<code>policy_context: PolicyContext</code>	Off-chain policy inputs, approval context, and risk labels.
<code>expiration: TransactionExpiration</code>	Sui transaction expiration/epoch constraints.

SuiIntentDigest

FIELD	DESCRIPTION
<code>intent_bytes: [u8; 3]</code>	Sui intent domain separator.
<code>transaction_data_bcs: Vec<u8></code>	Transaction data bound to the request.
<code>digest: [u8; 32]</code>	<code>Blake2b256(BCS(IntentMessage<TransactionData>))</code> .

Rule: recompute the digest independently before signing and before submit to prevent drift between builder, policy, and submitter.

LuvionSuiKey

FIELD	DESCRIPTION
<code>key_id: [u8; 32]</code>	Internal Luvion key identity.
<code>scheme: Ed25519</code>	Sui account-auth uses Ed25519.
<code>sui_public_key: [u8; 32]</code>	Aggregate Ed25519 public key.
<code>sui_address: SuiAddress</code>	<code>Blake2b256(0x00 public_key)</code> .
<code>committee_epoch: u64</code>	Luvion committee/key epoch.
<code>threshold: u16</code>	Target is 22.
<code>participants: Vec<NodeId></code>	Node set for the current key epoch.

SuiSignedEnvelope

FIELD	DESCRIPTION
<code>tx_bytes: Base64</code>	Transaction bytes required by the Sui execute API.
<code>signature: Base64</code>	<code>0x00 ed25519_signature aggregate_public_key</code> .
<code>tx_digest: TransactionDigest</code>	Independently computed and recorded before submit.
<code>request_type: ExecuteRequestType</code>	Local execution or effects-cert waiting strategy.

FrostSigningSession

FIELD	DESCRIPTION
<code>session_id: [u8; 32]</code>	Bound to <code>request_id</code> and digest.
<code>key_id: [u8; 32]</code>	Selects the aggregate key.
<code>participant_set: Vec<NodeId></code>	Sorted, deduplicated, deterministic encoding.
<code>nonce_commitments: Vec<NonceCommitment></code>	Round1 output, single-use only.
<code>signing_message: [u8; 32]</code>	Exact Sui intent digest used as the RFC 9591 FROST message slot.
<code>transcript_audit_hash: [u8; 32]</code>	Internal audit hash, independent of RFC 9591 binding-factor inputs.
<code>partial_signatures: Vec<PartialSignature></code>	Round2 output, verified individually before aggregation.

Binding factors are derived by RFC 9591 Section 4.4 from group public key, message hash, and commitment list. Luvion keeps binding-factor computation within the RFC 9591 input set; any domain separation is fixed by the Sui intent digest or upstream transaction intent.

7.2 Optional Move Objects

KeyRegistry has key

- Shared object only when on-chain key discovery, rotation state, or public audit is required.
- Fields: `id`, `current_key_version`, `current_sui_address`, `committee_hash`, `paused`, `admin_epoch`.

KeySnapshot has key + store

- Frozen as immutable after publication for audit of a key epoch.
- Fields: `id`, `key_version`, `sui_address`, `public_key`, `threshold`, `committee_hash`, `created_at_epoch`.

PolicyConfig has key

- Shared object only if policy mutation requires on-chain transparency.
- Fields: `id` , `policy_hash` , `limit_config` , `paused` , `admin_cap_id` .

PolicyAdminCap has key + store

- Address-owned capability used to mutate `KeyRegistry` and `PolicyConfig` .

CustodyVault has key

- Address-owned when a single Luvion-controlled address owns assets and policy is enforced off-chain.
- Shared only when multiple external addresses/contracts must deposit, withdraw, or update state through the same object. Move vaults control application-layer assets deposited into the vault and execute application-level policy; Sui account-auth is handled by the aggregate Ed25519 address.

SigningRequestObject has key + store

- Optional. Prefer off-chain request records unless on-chain request transparency is a product requirement.

8. Owned / Shared Boundary

Address-owned objects:

- Gas coins.
- Custody objects controlled by the aggregate Sui address.
- Admin/operator caps.
- Per-user receipts.
- Single-owner vaults.

Use when:

- Signing and submit need low latency.
- Only the Luvion aggregate address needs to use the object as transaction input.
- Policy, approval, and audit are primarily handled off-chain.

Risks:

- Assets from old accounts move into the new aggregate address through migration transactions.
- If gas owner differs from sender, an extra gas owner signature or sponsor flow is required.

Shared objects:

- Public key registry.
- Shared vault.
- Public request queue.

- Nonce/replay book.
- Policy state accessed by multiple users or contracts.

Use when:

- Key/policy state must be publicly discoverable.
- Multiple external actors must submit requests concurrently.
- Move-level application authorization is required.

Costs:

- Enters Sui's consensus path.
- Higher contention risk.
- More complex state migration, object lock, and stale object-ref handling.

Immutable objects:

- Published package object.
- Frozen key snapshots.
- Frozen policy snapshots.
- Audit anchors.

Boundary rules:

- Default to off-chain policy plus address-owned execution.
- Introduce shared objects only when product semantics require public concurrent access or on-chain transparency.
- A Move shared vault is application-level custody for deposited assets; Sui account-auth is handled by the Ed25519/FROST aggregate address.

9. State Migration and Exception Handling

9.1 Off-Chain Signing State Machine

Draft

- > ObjectsResolved
- > Simulated
- > PolicyApproved
- > FrostRound1
- > FrostRound2
- > SignatureAggregated
- > Submitted
- > Checkpointed

State descriptions:

STATE	DESCRIPTION
Draft	Intent, operations, objects, and policy context are being assembled.
ObjectsResolved	Object refs, gas coin, gas price/budget are resolved.
Simulated	Dry-run/dev-inspect returned expected effects and gas estimate.
PolicyApproved	Off-chain policy engine approved signing.
FrostRound1	Nodes submit nonce commitments.
FrostRound2	Nodes submit partial signatures.
SignatureAggregated	Sui Ed25519 signature is aggregated and locally verified.
Submitted	Transaction submitted to Sui fullnode/RPC.
Checkpointed	Effects confirmed and available for audit.

Exception handling:

EXCEPTION	TRANSITION	HANDLING
stale object ref	ObjectsResolved -> Draft	Re-fetch object version/digest and rebuild transaction.
insufficient gas budget	Simulated -> Draft	Adjust gas budget/payment and simulate again.
policy rejection	PolicyApproved -> Rejected	Record rejection reason and skip FROST.
Round1 timeout	FrostRound1 -> Retry	Select a new session and regenerate nonce commitments.
Round2 timeout	FrostRound2 -> Retry	Discard session; use fresh nonce.
invalid partial	FrostRound2 -> FailedFinal or replacement node	Create blame record and isolate node by policy.
duplicate commitment	FrostRound1 -> FailedFinal	Treat as protocol violation.
nonce reuse	FrostRound* -> FailedFinal	Treat as critical security event.
Sui submit reject	Submitted -> Draft	Check signature, object refs, epoch, gas, and sender.
Move abort	Submitted -> FailedFinal	Record effects and abort location.
object lock/ equivocation	Submitted -> OperationalIncident	Epoch-scoped operational incident; after manual resolution, re-pin object refs and return to Draft .

Mandatory checks:

- Signing layer uses the intent digest.
- Recompute the intent digest before signing and before submit.
- Verify `sender == SuiAddress::from(aggregate_public_key)`.
- Use fresh nonce material for every retry.
- Verify every partial signature before aggregation.
- Locally verify the final Ed25519 signature before submit.

9.2 Optional Move State Machines**KeyRegistry**

```
Active -> Rotating -> Active
Active -> Paused -> Active
Active -> Draining -> Retired
```

SigningRequestObject

```
Pending -> Approved -> Executed
Pending -> Rejected
Pending -> Expired
Approved -> Cancelled
```

CustodyVault

```
Active -> Paused -> Active
Active -> Draining -> Closed
```

10. Interface List

Traits and APIs in this section are proposed additions to be implemented in the Sui adapter, FROST backend, and optional Move package.

10.1 Internal Rust Traits

```

trait SuiTransactionResolver {
    fn resolve_inputs(&self, draft: SuiTxDraft) -> Result<SuiSigningRequest>;
    fn simulate(&self, request: &SuiSigningRequest) -> Result<SimulationReport>;
}

trait SuiIntentDigestBuilder {
    fn build_digest(&self, request: &SuiSigningRequest) -> Result<SuiIntentDigest>;
    fn verify_sender(&self, request: &SuiSigningRequest, key: &LuvionSuiKey) -> Result<()>;
}

trait SuiThresholdSigner {
    fn public_key(&self, key_id: [u8; 32]) -> Result<[u8; 32]>;
    fn sign_intent_digest(&self, request: SuiSigningRequest) -> Result<SuiSignedEnvelope>;
}

trait SuiSubmitter {
    fn execute(&self, envelope: SuiSignedEnvelope) -> Result<SuiExecutionReport>;
    fn wait_checkpoint(&self, digest: TransactionDigest) -> Result<CheckpointReport>;
}

trait FrostEd25519Backend {
    fn start_round1(&self, session: FrostSigningSession) -> Result<Vec<NonceCommitment>>;
    fn collect_round2(&self, session: FrostSigningSession) -> Result<Vec<PartialSignature>>;
    fn aggregate(&self, session: FrostSigningSession) -> Result<[u8; 64]>;
}

```

10.2 External Service API

ENDPOINT	DESCRIPTION
POST /sui/tx/build	Build PTB/TransactionData and resolve object inputs.
POST /sui/tx/simulate	Dry-run/simulate before policy approval.
POST /sui/tx/sign	Sign Sui intent digest through the Luvion FROST backend.
POST /sui/tx/submit	Submit signed transaction block.
GET /sui/tx/{digest}	Query effects, checkpoint, and failure reason.
GET /sui/keys/{key_id}	Expose Sui public key, address, and committee epoch.
GET /sui/requests/{request_id}	Query request state, participants, and error records.

10.3 Sui Network Interfaces

- Prefer Sui gRPC or the current stable SDK path for execution, dry-run, and object reads.
- Use GraphQL RPC or a custom indexer for read-heavy indexing queries.

- Sui docs mark JSON-RPC as deprecated and require migration to gRPC or GraphQL RPC by July 2026; new integration work uses gRPC/SDK plus GraphQL/indexer as the long-term interface.

10.4 Optional Move Entry Functions

```
public entry fun init_registry(admin: address, ctx: &mut TxContext)
public entry fun rotate_key(registry: &mut KeyRegistry, cap: &PolicyAdminCap, new_key: vector<u8>)
public entry fun pause(registry: &mut KeyRegistry, cap: &PolicyAdminCap)
public entry fun unpause(registry: &mut KeyRegistry, cap: &PolicyAdminCap)
public entry fun create_vault(config: &PolicyConfig, ctx: &mut TxContext)
public entry fun submit_request(vault: &mut CustodyVault, ...)
public entry fun mark_executed(request: &mut SigningRequestObject, digest: vector<u8>)
```

11. Implementation Phases

Phase 0: version pinning and test vectors

- Initial Sui pin: `MystenLabs/sui` tag `mainnet-v1.70.2`, commit `6d4ec0b0621dd9555753c9ecd5be021b25a0d267`.
- Rust adapter uses `sui-sdk`, `sui-types`, `shared-crypto`, and `sui-json-rpc-types` from the same git revision; gRPC schema uses the same tag.
- Build Ed25519 address derivation test vectors.
- Build `BCS(IntentMessage<TransactionData>)` digest test vectors.
- Build `0x00 || sig || pk` local verification tests.

Phase 1: threshold Ed25519/FROST backend

- Implement DKG or controlled key ceremony.
- Define keygen path: RFC 9591 covers trusted-dealer keygen; testnet may use controlled import ceremony, while the production release gate is audited Pedersen/FROST DKG or an independently reviewed secure key ceremony.
- Implement nonce commitment round.
- Implement partial signature round.
- Implement partial verification, blame, and duplicate commitment checks.
- Implement aggregate signature output and local Ed25519 verification.

Phase 2: Sui adapter

- Resolve object refs and gas.
- Build `TransactionData`.
- Simulate/dry-run.
- Compute intent digest.
- Call FROST backend for signing.
- Submit and watch effects/checkpoints.

Phase 3: key/policy persistence

- Build off-chain key registry.
- Record key epoch, committee hash, and participant set.
- Record request lifecycle, signing transcript hash, failure records, and blame records.

Phase 4: optional Move package

- Implement only if on-chain discoverability or shared vault semantics are required.
- Document that shared vaults only control deposited assets.
- Benchmark shared-object contention, Move policy execution gas, and object lock / stale reference failure modes.

Phase 5: hardening

- Object-ref stale tests.
- Sponsored gas tests.
- Equivocation/lock recovery tests.
- Invalid partial / duplicate commitment / nonce reuse tests.
- Insufficient quorum / retry budget tests.
- Testnet end-to-end execution.

12. Verification Checklist

Cryptography:

- FROST aggregate signature is accepted by Sui fastcrypto ZIP215 Ed25519 verifier.
- FROST aggregate signature passes fixed Sui fastcrypto ZIP215 test vectors.
- Any invalid partial is detected and attributed.
- Retry uses fresh nonce material.
- Participant ordering is deterministic.
- Aggregate public key and Sui address derivation match.

Sui compatibility:

- Intent digest matches Sui SDK/source test vectors.
- `tx_bytes` matches `TransactionData` BCS encoding.
- Signature envelope format is `0x00 || signature || public_key`.
- Sender/gas owner/sponsor rules are covered.
- Stale object refs trigger transaction rebuild.

System reliability:

- Signature production requires quorum.
- Node timeout retry uses a new session.
- Blame records are auditable.

- Submit failure and effects failure are recorded separately.
- Checkpoint watcher handles delayed confirmation and query failure.

13. Main Technical Risks

RISK	IMPACT	CONTROL
nonce reuse	Aggregate private key recovery and full key compromise	Session-level nonce lifecycle, persistent replay guard, fresh nonce on retry, critical security alerting.
FROST ciphersuite semantic drift from Sui Ed25519 verifier	Signature submission failure	Use RFC 9591 FROST (Ed25519, SHA-512) and align local/on-chain verification on Sui fastcrypto ZIP215 semantics.
DKG/import ceremony audit gap	Insufficient key-material production assurance	Use RFC 9591 trusted-dealer keygen as baseline; testnet may use controlled import ceremony, while the production release gate is audited DKG or independently reviewed secure key ceremony.
ZIP215/RFC8032 local verifier drift	Local self-verification and on-chain verification diverge	Pin local self-verification to Sui fastcrypto verifier/ mode.
threshold ML-DSA standardization maturity	PQ backend production timing depends on standards and audit	Sui account-auth compatibility uses Ed25519/FROST; threshold ML-DSA continues as the long-term post-quantum migration layer and core technical moat.
stale object refs	Submit rejection or object lock conflict	Recompute and rebuild across resolve/simulate/sign/ submit boundaries.
missing sponsored tx authorization	Signature verification failure	Separate sender signature and gas owner signature responsibilities.
shared object contention / object lock / stale reference	Higher latency, lock conflict, or submission failure	Default to address-owned; benchmark shared-path contention, Move policy gas, and object lock/stale reference failure modes.
Move vault / account-auth boundary confusion	Asset-control semantics are classified incorrectly	Keep account-auth and application-level vault paths explicitly separate.

14. Reviewed Sources

- Current Luvion code: `src/params.rs` , `src/poly/*` , `src/lagrange.rs` , `src/signing.rs` , `src/dkg.rs` , `src/share_io.rs` , `src/vrf.rs` , `src/network.rs` , `src/p2p.rs` , `src/orchestrator.rs` , `src/bin/_cluster/mod.rs` , `src/bin/node.rs` , `src/bin/cluster_sign.rs` .
- Sui docs:
 - <https://docs.sui.io/concepts/cryptography/transaction-auth/intent-signing>
 - <https://docs.sui.io/concepts/cryptography/transaction-auth/signatures>
 - <https://docs.sui.io/concepts/cryptography/transaction-auth/multisig>
 - <https://docs.sui.io/concepts/object-model>
 - <https://docs.sui.io/references/sui-api>
- FROST standard: <https://www.rfc-editor.org/rfc/rfc9591>
- fastcrypto Ed25519 ZIP215 verifier: <https://github.com/MystenLabs/fastcrypto/blob/main/fastcrypto/src/ed25519.rs>
- FIPS 204 ML-DSA: <https://csrc.nist.gov/pubs/fips/204/final>
- Threshold ML-DSA research reference: <https://eprint.iacr.org/2025/1163.pdf>
- Sui source: `MystenLabs/sui` ; reviewed paths include `crates/sui-types/src/crypto.rs` , `crates/sui-types/src/base_types.rs` , `crates/sui-types/src/signature.rs` , `crates/sui-types/src/transaction.rs` , `crates/sui-types/src/multisig.rs` , `crates/sui-framework/packages/sui-framework/sources/transfer.move` .